When to Compile a Computer-Use Agent? Measuring Payback and Making Compilation Decisions for Token Efficiency

Yulong Ming, Jie Xu, Zihan Wu, Xiaohua Jia

Paper version: arXiv 2610.02932v1 · CUA Radar Reel poster, not made or endorsed by the authors

Problem

> *Does compiling a repeated GUI procedure into a program pay back its token cost?*

Attempts can need **repair and still fail**, and future reuse is unknown.

Motivation: three gaps

1. **Uncertain cost.** Repair, or no usable program at all.
2. **Unknown reuse.** Tasks stop; GUI drift breaks programs.
3. **No shared count.** Audits report 4 to 17 uses.

Evaluation setup

AndroidWorld · 4 families · LibreOffice · 2 families · WebArena Reddit · 1 · GLM 5.3 Flash · DeepSeek Flash · Qwen 3.8 Flash

- **Measured:** compile and serve token cost. **Simulated:** decision rules on 2 × 300 conditions + 4 logs.
- **Fresh:** 30 bindings per program; logs give arrival order only.

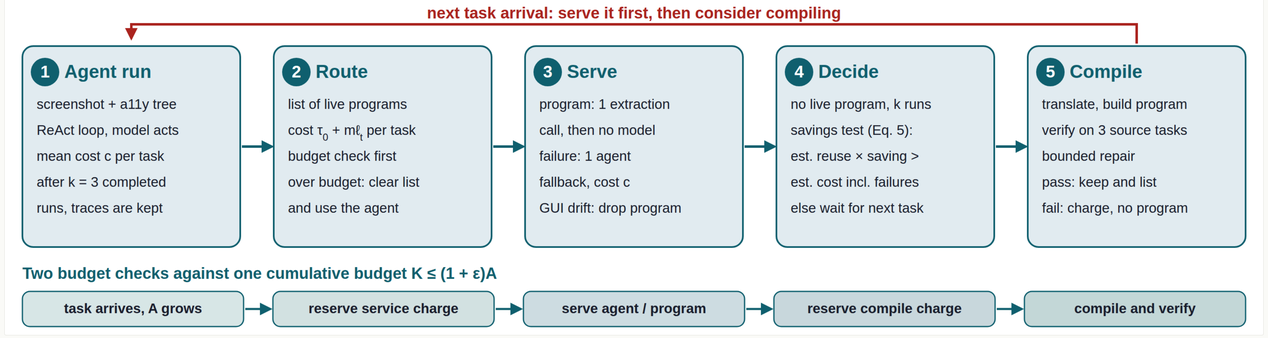
Our read (CUA Radar)

69	★★	offline
score of 100	band	evidence level

- **Reports:** bootstrap CIs; same-charge baselines; cost; ablation; code. Online eval and held-out: partial.
- **Flags:** AP8: “not independent confirmation”.
- Pattern C19. Authors’ numbers, not rerun.

Method: PACE serves first, then decides whether to compile

- **Sees / acts:** screenshot + a11y tree (DOM on web) per step; ReAct loop, or a compiled program after one input-extraction call.
- **Mechanism:** compile only if estimated savings beat the estimated cost, failed attempts included.
- **Signal:** token charges and verification outcomes; no training. A budget caps spend.
- **Guarantee:** under stated cost assumptions, total cost stays at most 1 + ε times agent-only after each arrival.



Our redraw (not the paper’s figure): the serve-then-decide loop and the two budget checks.

Key results: simulated token cost (× PACE)

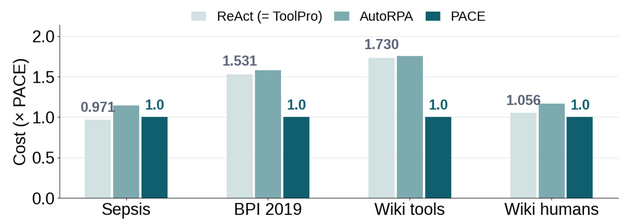


Table 3, redrawn: GLM 5.3 Flash, 10 paired runs; ToolPro equals ReAct.

Mean 17.34% below ReAct; Sepsis: ReAct 0.971 × PACE

SO WHAT → *Offline simulation on measured costs, ε = 0.25. AutoRPA costs more than PACE in every log condition; ReAct is cheaper on Sepsis (0.971). The logs give task frequencies and arrival order, not live GUI tasks.*

Ablation: simulated cost on recorded logs

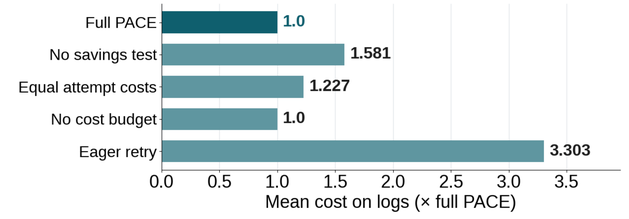


Table 4, redrawn: Logs mean, GLM 5.3 Flash column; each variant relative to full PACE.

SO WHAT → *No savings test: +52.68% mean cost on logs (all models). Equal attempt costs: +23.34%. Eager retry: 3.303 to 4.922 × PACE by model. No budget: 0.0013% on logs, but the largest tested grid ratio to ReAct rises from 1.2468 to 1.7563. The paper used these grids while developing PACE, so they are not independent confirmation.*

Headline Numbers

17.34%
simulated token saving
vs ReAct, recorded logs

- **24.9%** vs AutoRPA adaptation
- **9 of 12** conditions cheaper
- **2–16 uses** payback, compile only

The paper’s own controls

CONTROL	PAPER REPORTS	COMPARED WITH
Sim. on logs (offline)	17.34% [14.96, 19.61]	cheaper in 9 of 12
Matched agent replays	saving share 0.952–0.999	agent 157 / prog 178 of 180
Live Calendar, 12 pairs	program + fallback 11/12	agent 7/12 (4 interrupted)
Costly + frequent tasks	1.015 × ReAct (mean)	about 1.5% above ReAct

Takeaway

17.34% lower simulated token cost than ReAct on recorded logs; spend capped at 1 + ε times agent-only.

offline, simulated · payback 2–16 uses · 9 of 12 conditions · ε = 0.25

Scan to Read

Paper · Code